Roteiro das aulas — ENCOMP Agenda

Material do instrutor. Dois minicursos: **Backend (3h30)** e **Frontend (4h)**.

Projeto: agenda de eventos · API Node/Express/SQLite · front HTML/CSS/JS · API publicada pelo instrutor no Render.

Pré-requisitos

Instrutor

- [ ] Repositório no GitHub
- [ ] Conta Render pronta (deploy da API é seu)
- [ ] Slides de teoria (backend e frontend)
- [ ] URL pública anotada ao fim do dia 1

Alunos (PC)

- [ ] Node.js LTS (18+) — `node -v` / `npm -v`
- [ ] Git
- [ ] VS Code
- [ ] Extensão Thunder Client
- [ ] Live Server (opcional) ou uso de `npx serve`
- [ ] Navegador (Chrome/Edge)

Alunos **não** precisam de conta Render.

Dia 1 — Backend (3h30)

Bloco 1 — Conceitos (0:00–0:20)

Slides de teoria.

- O que é API e por que separar front/back
- HTTP: método, URL, headers, body, status
- JSON
- REST: recurso + verbo
- Demo rápida no Thunder Client

Saída: vocabulário alinhado.

Bloco 2 — Setup Express (0:20–0:45)

- `npm init -y` , `express` , `cors`
- `index.js` + `app.listen`
- `GET /health` → `{ "ok": true }`
- Testar no Thunder Client

Saída: servidor local ok.

Bloco 3 — SQLite + leitura (0:45–1:25)

- Modelo `evento` (campos)
- `better-sqlite3` + `db.js` (tabela + seed)
- Mostrar `data/agenda.db`
- `GET /eventos` e `GET /eventos/:id`

Saída: API lê do banco.

Bloco 4 — Criação + validação (1:25–2:05)

- `POST /eventos` + `express.json()`
- Validar `titulo`, `data`, `local` → 400
- `id` gerado; status padrão `agendado`
- Resposta 201; conferir persistência

Saída: criar funciona.

Bloco 5 — Atualizar e excluir (2:05–2:35)

- `PUT /eventos/:id`
- `DELETE /eventos/:id` → 204
- 404 consistente

Saída: CRUD local completo.

Bloco 6 — CORS, env, pastas (2:35–2:55)

- `cors()`, `PORT`, `routes/eventos.js`, `.gitignore`

Saída: pronto para publicar.

Bloco 7 — Deploy (instrutor) (2:55–3:30)

- Push GitHub → Web Service Render (root: `backend`)
- Testar `/health` e CRUD na URL pública
- **Anotar URL no quadro** (base do dia 2)

Entrega dia 1: API pública com CRUD.

Dia 2 — Frontend (4h)

Premissa: `API_BASE` = URL do Render no quadro. Front com Live Server / `npx serve` (não usar `file://`).

Bloco 1 — HTML (0:00–0:25)

- Header, formulário, lista, mensagens
- Campos: título, data, local, status, descrição

Saída: página estática.

Bloco 2 — CSS (0:25–1:05)

- Layout mobile-first, tipografia, itens da lista
- Estados: loading, erro, vazio

Saída: UI legível.

Bloco 3 — Listar (1:05–1:50)

- `API_BASE` + `fetch` GET
- Renderizar no DOM
- Vazio / erro de rede

Saída: dados reais na tela.

Bloco 4 — Criar (1:50–2:35)

- Submit + `preventDefault`
- POST JSON
- Limpar form + recarregar lista

- Mostrar erro 400

Saída: cadastro pelo browser.

Bloco 5 — Editar e excluir (2:35–3:15)

- Editar preenche form + PUT
- Excluir + DELETE (+ `confirm`)
- Cancelar edição

Saída: CRUD no front.

Bloco 6 — UX (3:15–3:40)

- Loading, mensagens, CORS na prática

Bloco 7 — Encerramento (3:40–4:00)

- Opcional: filtro por status
- Recap: request → API → JSON → DOM
- Próximos passos (auth, framework) sem implementar

Entrega dia 2: front consumindo a API publicada.

Checklist de sucesso

Dia 1

- [ ] `/health` público responde
- [ ] CRUD de `/eventos` ok no Thunder Client (URL Render)
- [ ] URL no quadro

Dia 2

- [ ] Lista carrega da API
- [ ] Criar / editar / excluir refletem na API
- [ ] Erro da API aparece na interface

Endpoints (cola rápida)

Método	Caminho	Ação
GET	`/health`	health
GET	`/eventos`	listar
GET	`/eventos/:id`	detalhar
POST	`/eventos`	criar
PUT	`/eventos/:id`	atualizar
DELETE	`/eventos/:id`	remover

Campos obrigatórios no body: `titulo`, `data`, `local`.